Experiment 5

Title

To apply JavaScript array methods and object handling by developing a Shopping Cart Total Calculator with Discount Logic.

Software/Tools Required

- Visual Studio Code
- Google Chrome
- HTML5
- CSS3
- JavaScript (ES6)

Theory

Arrays in JavaScript are used to store multiple values in a single variable. Array methods like push(), forEach(), reduce() help perform operations on data efficiently.

Objects store information in key-value pairs.

In this experiment, the user enters product details. Each product is stored as an object inside an array. The program calculates the total bill, applies a discount, and displays the final amount.

Code

index.html

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Shopping Cart Calculator</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">
```

```
<h1>Shopping Cart Calculator</h1>
<label>Customer Name</label>
<input type="text" id="customer" placeholder="Enter Customer Name">
<label>Product 1 Price</label>
<input type="number" id="p1" placeholder="Enter Price">
<label>Product 2 Price</label>
<input type="number" id="p2" placeholder="Enter Price">
<label>Product 3 Price</label>
<input type="number" id="p3" placeholder="Enter Price">
<button onclick="generateBill()">Generate Bill</button>
<div id="result"></div>
</div>
<script src="script.js"></script>
</body>
</html>
```

style.css

```
*{
margin:0;
padding:0;
box-sizing:border-box;
font-family:Arial, Helvetica, sans-serif;
}
body{
background:linear-gradient(135deg,#eef3fb,#d7e5ff);
display:flex;
justify-content:center;
align-items:center;
min-height:100vh;
padding:30px;
}
.container{
width:620px;
background:#fff;
```

```
padding:35px;
border-radius:18px;
box-shadow:0 10px 30px rgba(0,0,0,.15);
}
h1{
text-align:center;
color:#0b4ea2;
margin-bottom:25px;
font-size:36px;
}
label{
display:block;
margin-top:18px;
font-weight:bold;
font-size:18px;
}
input{
width:100%;
padding:14px;
margin-top:8px;
border:1px solid #ccc;
border-radius:8px;
font-size:17px;
}
input:focus{
outline:none;
border-color:#0b4ea2;
box-shadow:0 0 8px rgba(11,78,162,.3);
}
button{
width:100%;
margin-top:30px;
padding:15px;
background:#0b4ea2;
```

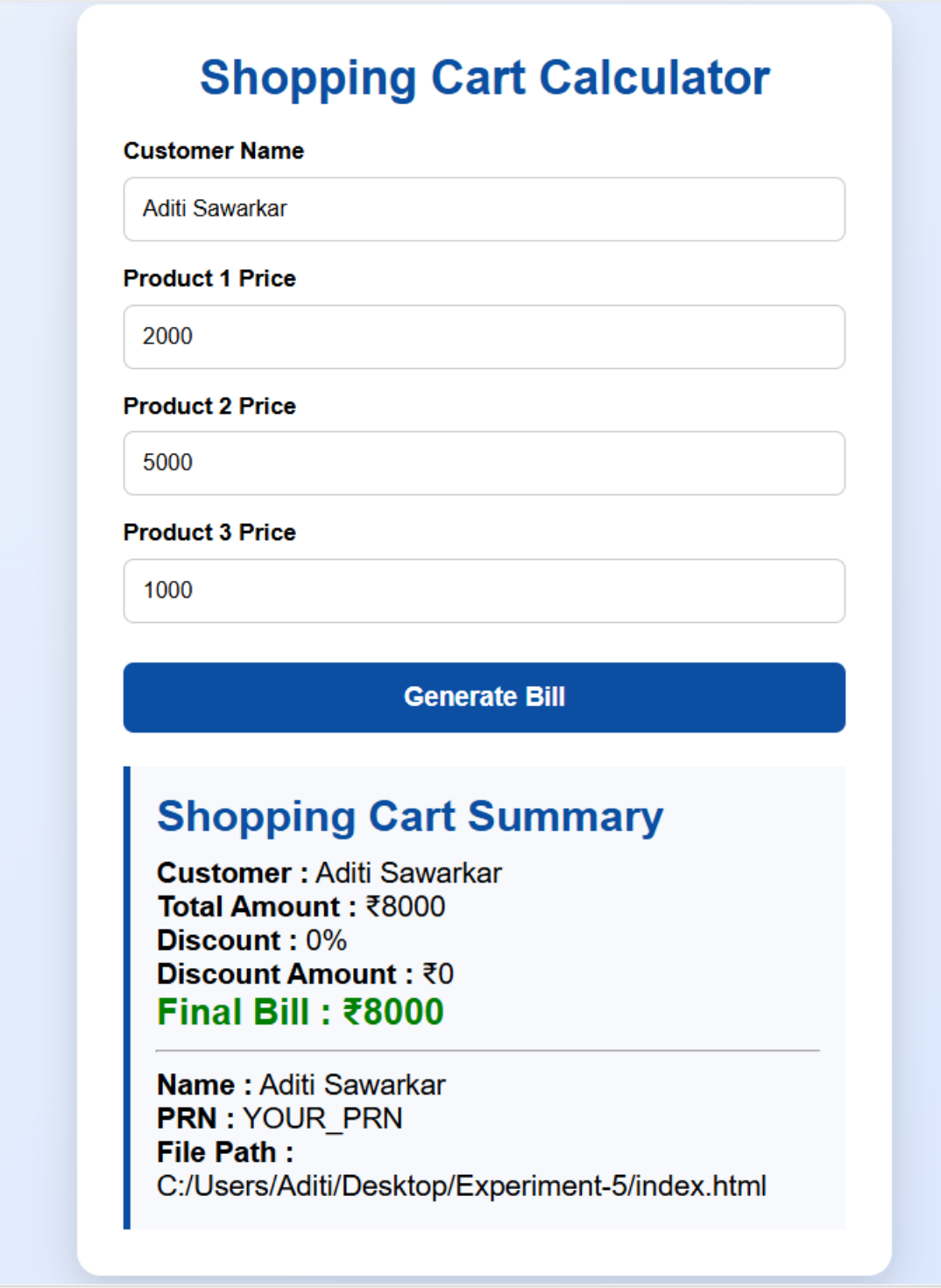
```
color:white;
border:none;
font-size:20px;
font-weight:bold;
border-radius:8px;
cursor:pointer;
transition:.3s;
}
button:hover{
background:#06397b;
}
#result{
margin-top:25px;
border-left:6px solid #0b4ea2;
background:#f7f9fc;
padding:20px;
font-size:22px;
display:none;
}
#result h2{
color:#0b4ea2;
margin-bottom:10px;
}
.green{
color:green;
font-size:28px;
font-weight:bold;
}
hr{
margin:12px 0;
}
```

script.js

```
function generateBill(){
```

```
let customer=document.getElementById("customer").value;
let p1=Number(document.getElementById("p1").value);
let p2=Number(document.getElementById("p2").value);
let p3=Number(document.getElementById("p3").value);
if(customer==""){
alert("Enter Customer Name");
return;
}
let total=p1+p2+p3;
let discount=0;
if(total>=100000){
discount=20;
}
else if(total>=50000){
discount=15;
}
else if(total>=20000){
discount=10;
}
let discountAmount=total*discount/100;
let finalBill=total-discountAmount;
document.getElementById("result").style.display="block";
document.getElementById("result").innerHTML=`
<h2>Shopping Cart Summary</h2>
<p><b>Customer :</b> ${customer}</p>
<p><b>Total Amount :</b> ₹${total}</p>
<p><b>Discount :</b> ${discount}%</p>
<p><b>Discount Amount :</b> ₹${discountAmount}</p>
<p class="green">Final Bill : ₹${finalBill}</p>
<hr>
<p><b>Name :</b> Aditi Sawarkar</p
<p><b>PRN :</b> YOUR_PRN</p>
<p><b>File Path :</b></p>
<p>C:/Users/Aditi/Desktop/Experiment-5/index.html</p>
`;
}
```

Output



Conclusion

Thus, the program was successfully executed to demonstrate JavaScript arrays, objects, array methods (push(), forEach(), reduce()), and discount calculation. The Shopping Cart application accepted product details, generated the total bill, applied discount logic, and displayed the final payable amount successfully.

Experiment 6

Title

Use string functions and regex for email validation, data extraction, and text analysis.

Software/Tools Required

- Visual Studio Code (VS Code)
- Google Chrome / Microsoft Edge
- HTML5
- CSS3
- JavaScript (ES6)

Theory

Strings in JavaScript are used to store and manipulate text data. JavaScript provides many built-in string functions such as **toUpperCase()**, **toLowerCase()**, **trim()**, **split()**, and **length** to perform various text operations.

Regular Expressions (Regex) are patterns used to search, validate, and extract data from strings. They are commonly used for tasks such as email validation, phone number validation, and data extraction.

In this experiment, the user enters a name, email address, and text. The program validates the email using a regular expression, counts the number of words and characters, extracts numeric values from the text, and converts the text to both uppercase and lowercase. This demonstrates the practical use of JavaScript string functions and regular expressions in web applications.

Code

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Smart Text Analyzer</title>
<link rel="stylesheet" href="style.css">
</head>
```

```
<body>
<div class="container">
<h1>✉ Smart Text Analyzer</h1>
<label>Name</label>
<input type="text" id="name" placeholder="Enter Your Name">
<label>Email Address</label>
<input type="text" id="email" placeholder="Enter Email">
<label>Enter Text</label>
<textarea id="text" rows="6"
placeholder="Write something here..."></textarea>
<button onclick="analyzeText()">
Analyze Text
</button>
<div id="result"></div>
</div>
<script src="script.js"></script>
</body>
</html>
```

style.css

```
*{
margin:0;
padding:0;
box-sizing:border-box;
font-family:Arial,sans-serif;
}
body{
background:linear-gradient(135deg,#6a11cb,#2575fc);
display:flex;
justify-content:center;
align-items:center;
min-height:100vh;
padding:30px;
}
.container{
```

```
width:700px;
background:white;
padding:35px;
border-radius:20px;
box-shadow:0 15px 35px rgba(0,0,0,.3);
}
text-align:center;
color:#1e40af;
margin-bottom:25px;
}
label{
display:block;
margin-top:18px;
font-weight:bold;
font-size:18px;
}
input,textarea{
width:100%;
padding:14px;
margin-top:8px;
border:1px solid #ccc;
border-radius:10px;
font-size:16px;
}
textarea{
resize:none;
}
button{
width:100%;
margin-top:25px;
padding:15px;
background:#2563eb;
color:white;
border:none;
```

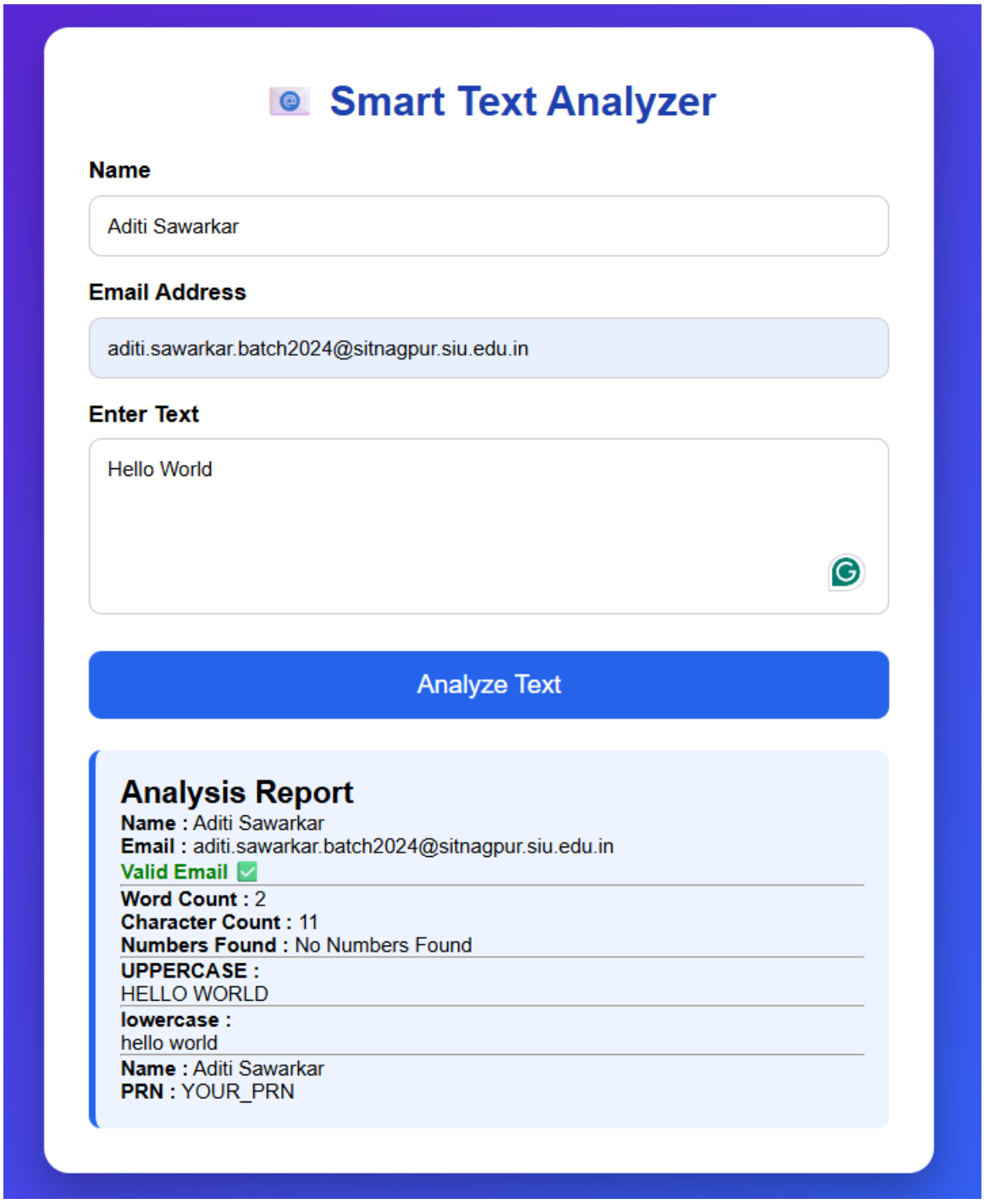
```
border-radius:10px;
font-size:20px;
cursor:pointer;
}
button:hover{
background:#1d4ed8;
}
#result{
margin-top:25px;
padding:20px;
background:#eef4ff;
border-left:6px solid #2563eb;
border-radius:10px;
display:none;
}
.valid{
color:green;
font-weight:bold;
}
.invalid{
color:red;
font-weight:bold;
}
```

script.js

```
function analyzeText(){
let name=document.getElementById("name").value.trim();
let email=document.getElementById("email").value.trim();
let text=document.getElementById("text").value;
if(name=="" || email=="" || text==""){
alert("Please fill all fields");
return;
}
let emailPattern=/^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$/;
```

```
let emailStatus=emailPattern.test(email)
?"<span class='valid'>Valid Email ✔</span>"
:"<span class='invalid'>Invalid Email ✖</span>";
let words=text.trim().split(/\s+/).filter(word=>word!="");
let wordCount=words.length;
let characterCount=text.length;
let upper=text.toUpperCase();
let lower=text.toLowerCase();
let numbers=text.match(/\d+/g);
numbers=numbers?numbers.join(", "):"No Numbers Found";
document.getElementById("result").style.display="block";
document.getElementById("result").innerHTML=`
<h2>Analysis Report</h2
<p><b>Name :</b> ${name}</p>
<p><b>Email :</b> ${email}</p>
<p>${emailStatus}</p>
<hr>
<p><b>Word Count :</b> ${wordCount}</p>
<p><b>Character Count :</b> ${characterCount}</p>
<p><b>Numbers Found :</b> ${numbers}</p>
<hr>
<p><b>UPPERCASE :</b><br>${upper}</p>
<hr>
<p><b>lowercase :</b><br>${lower}</p>
<hr>
<p><b>Name :</b> Aditi Sawarkar</p>
<p><b>PRN :</b> YOUR_PRN</p>
`;
}
```

Output



Conclusion

Thus, the program was successfully executed to demonstrate the use of JavaScript string functions and Regular Expressions (Regex). The application successfully validated the email address, counted words and characters, extracted numbers from the entered text, and converted the text into uppercase and lowercase. Hence, the objectives of the experiment were achieved successfully.

Experiment No. 7

Title

Perform DOM traversal and updates; develop a To-Do List app with add, edit, delete features.

Software/Tools Required

- Visual Studio Code (VS Code)
- Google Chrome / Microsoft Edge
- HTML5
- CSS3
- JavaScript (ES6)

Theory

The Document Object Model (DOM) is a programming interface for HTML documents. It allows JavaScript to access, modify, add, and delete HTML elements dynamically.

DOM methods such as getElementById(), createElement(), appendChild(), and remove() are used to manipulate webpage content. Event handling allows user actions such as button clicks to trigger specific functions.

In this experiment, a To-Do List application is developed where users can add tasks, mark tasks as completed, edit tasks, and delete tasks. This demonstrates dynamic DOM manipulation and event handling in JavaScript.

Code

index.html

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>To-Do List App</title>

<link rel="stylesheet" href="style.css">

</head>
```

```
<body>
<div class="container">
<h1>📝 Smart To-Do List</h1>
<input type="text" id="taskInput"
placeholder="Enter a task">
<button onclick="addTask()">Add Task</button>
<ul id="taskList"></ul>
</div>
<script src="script.js"></script>
</body>
</html>
```

style.css

```
*{
margin:0;
padding:0;
box-sizing:border-box;
font-family:Arial,sans-serif;
}
body{
background:linear-gradient(135deg,#667eea,#764ba2);
display:flex;
justify-content:center;
align-items:center;
min-height:100vh;
padding:30px;
}
```

```
.container{
width:650px;
background:white;
padding:30px;
border-radius:20px;
box-shadow:0 10px 30px rgba(0,0,0,.3);
}
h1{
text-align:center;
color:#4c1d95;
margin-bottom:20px;
}
input{
width:100%;
padding:14px;
border:1px solid #ccc;
border-radius:10px;
font-size:16px;
}
button{
width:100%;
margin-top:15px;
padding:14px;
background:#4c1d95;
color:white;
border:none;
```

```
border-radius:10px;
font-size:18px;
cursor:pointer;
}
button:hover{
background:#311b92;
}
ul{
margin-top:20px;
list-style:none;
li{
background:#f3f4f6;
padding:15px;
margin-top:10px;
border-radius:10px;
display:flex;
justify-content:space-between;
align-items:center;
}
.completed{
text-decoration:line-through;
color:green;
font-weight:bold;
}
.action-btn{
padding:8px 12px;
```

```
margin-left:5px;

border:none;

border-radius:6px;

color:white;

cursor:pointer;

}

.edit{

background:#f59e0b;

}

.delete{

background:#ef4444;

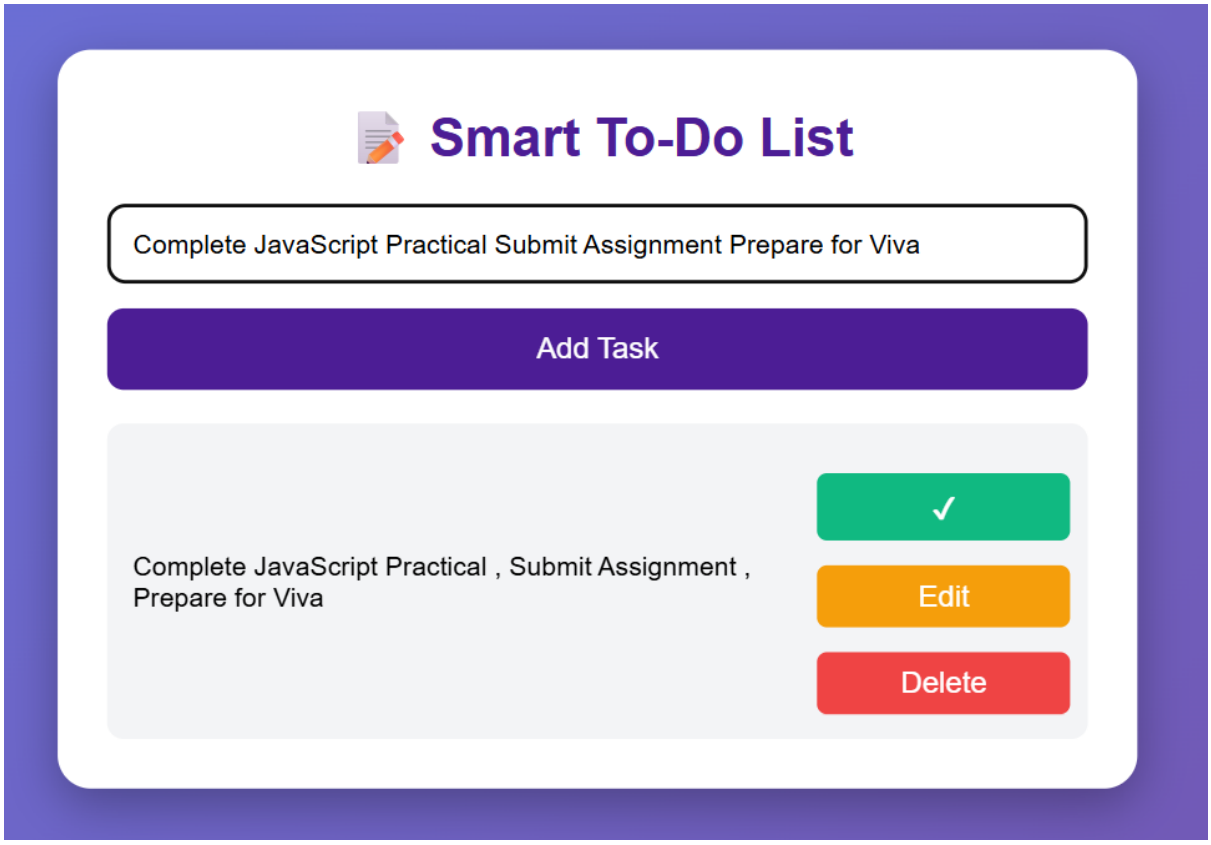
}

.done{

background:#10b981;

}
```

`Output


Smart To-Do List
Complete JavaScript Practical Submit Assignment Prepare for Viva
Add Task
Complete JavaScript Practical , Submit Assignment , Prepare for Viva
✓
Edit
Delete